



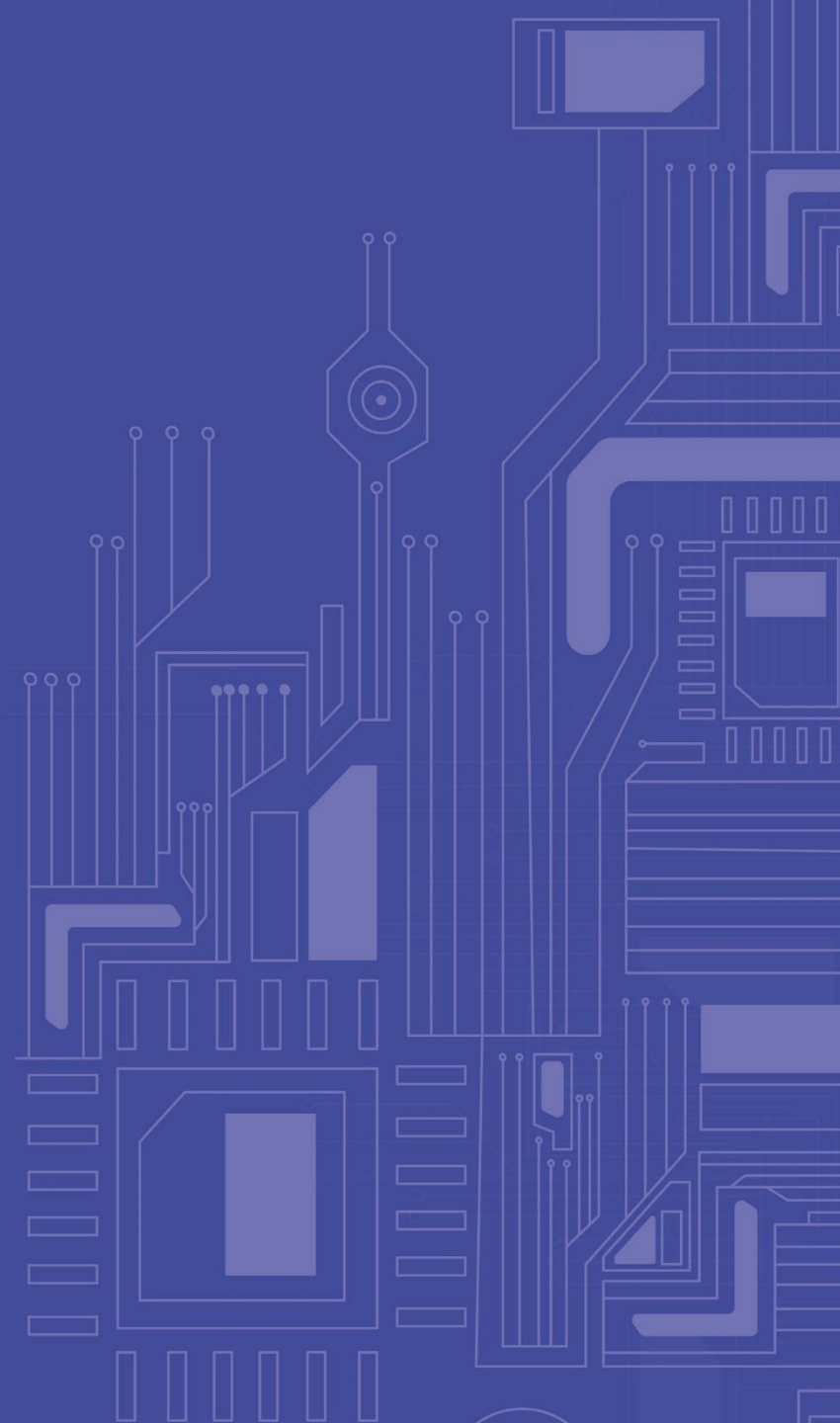
МИНОБРНАУКИ
РОССИИ



Передовые
инженерные
школы

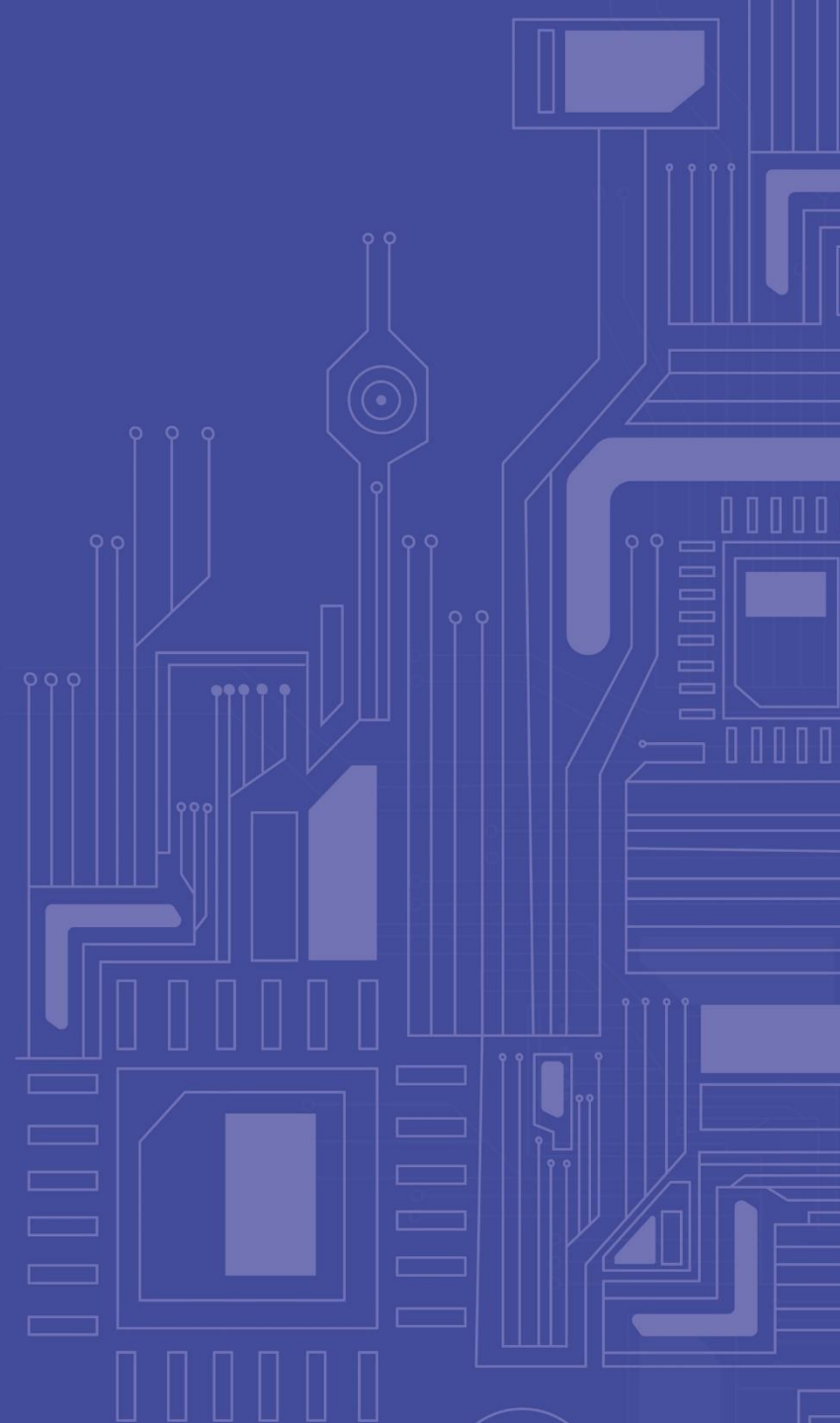
СИСТЕМЫ УПРАВЛЕНИЯ БАЗАМИ ДАННЫХ

Лекция 13



- Базовые средства манипулирования данными
- Рекурсивные запросы
- Оконные функции
- Пример рекурсии из новейшей практики (PostgreSQL)

БАЗОВЫЕ СРЕДСТВА МАНИПУЛИРОВАНИЯ ДАННЫМИ



К базовым средствам манипулирования данными языка SQL относятся «поисковые» варианты операторов **UPDATE** и **DELETE**. Эти варианты называются поисковыми, потому что при задании соответствующей операции задается логическое условие, налагаемое на строки адресуемой оператором таблицы, которые должны быть подвергнуты модификации или удалению. Кроме того, в такую категорию языковых средств входит оператор **INSERT**, позволяющий добавлять строки в существующие таблицы.

Оператор INSERT для вставки строк в существующие таблицы

Общий синтаксис оператора INSERT выглядит следующим образом:

```
INSERT INTO table_name
    { [ (column_comma_list) ] query_expression
    | DEFAULT VALUES
```

Даже если ограничиться простейшей составляющей этой конструкции (simple_table), то мы имеем следующие возможности:

```
simple_table ::= query_specification
    | table_value_constructor
    | TABLE table_name
```

```
query_expression ::= [ with_clause ] query_expression_body
query_expression_body ::= { non_join_query_expression
    | joined_table }
non_join_query_expression ::= non_join_query_term
    | query_expression_body
    { UNION | EXCEPT } [ ALL | DISTINCT ]
    [ corresponding_spec ] query_term
query_term ::= non_join_query_term | joined_table
non_join_query_term ::= non_join_query_primary
    | query_term INTERSECT [ ALL | DISTINCT ]
    [ corresponding_spec ] query_primary
query_primary ::= non_join_query_primary | joined_table
non_join_query_primary ::= simple_table
    | (non_join_query_expression)
simple_table ::= query_specification
    | table_value_constructor
    | TABLE table_name
corresponding_spec ::= CORRESPONDING
    [ BY column_name_comma_list ]
```

ВСТАВКА ВСЕХ СТРОК УКАЗАННОЙ ТАБЛИЦЫ



Стандарт допускает вставку в указанную таблицу всех строк некоторой другой таблицы (вариант `table_name`). Эта другая таблица может быть как базовой, так и представляемой.

- В определении представления не должны присутствовать ссылки на таблицу, в которую производится вставка.
- При использовании данного варианта оператора вставки число столбцов вставляемой таблицы должно совпадать с числом столбцов таблицы, в которую производится вставка, или с числом столбцов, указанных в списке `column_commalist`, если этот список задан.
- Типы данных соответствующих столбцов вставляемой таблицы и таблицы, в которую производится вставка, должны быть совместимыми.
- Если в операции задан список `column_commalist` и в нем содержатся не все имена столбцов таблицы, в которую производится вставка, то в оставшиеся столбцы во всех строках заносятся значения столбцов по умолчанию.
- Если для какого-либо из оставшихся столбцов значение по умолчанию не определено, при выполнении операции вставки фиксируется ошибка.

ПРИМЕР Предположим, что в базе данных EMP-DEPT-PRO имеется еще одна промежуточная таблица EMP_TEMP, в которой временно хранятся данные о служащих, проходящих испытательный срок. Пусть эта таблица имеет следующий заголовок:

EMP_TEMP:

EMP_NO	:	EMP_NO
EMP_NAME	:	VARCHAR
EMP_BDATE	:	DATE



```
INSERT INTO EMP (EMP_NO, EMP_NAME, EMP_BDATE) TABLE EMP_TEMP;
```

В столбцах EMP_NO, EMP_NAME, EMP_BDATE будут данные, взятые из таблицы EMP_TEMP, а в столбцах EMP_SAL, DEPT_NO, PRO_NO будут значения по умолчанию.

EMP_NO не должен нарушать ограничения целостности первичного ключа.

ВСТАВКА ЯВНО ЗАДАННОГО ЧИСЛА СТРОК



Набор вставляемых строк задается явно с использованием синтаксической конструкции `table_value_constructor`.

```
table_value_constructor ::=
    VALUES row_value_constructor_comma_list
row_value_constructor ::= row_value_constructor_element
    | [ ROW ] (row_value_constructor_element_comma_list)
    | row_subquery
row_value_constructor_element ::= value_expression
    | NULL | DEFAULT
```

Варианты вставки:

```
INSERT INTO EMP
ROW (2445, 'Brown', '1985-04-08', 16500.00, 630, 772);
```

```
INSERT INTO EMP
ROW ( 2445, DEFAULT, NULL, DEFAULT, NULL, NULL);
```

<=>

```
INSERT INTO EMP (EMP_NO) 2445;
```

Одной из разновидностей `value_expression_primary` является **scalar_subquery**

```
INSERT INTO EMP VALUES
    ROW (2445, (SELECT EMP_NAME
                FROM EMP
                WHERE EMP_NO = 2555),
        '1985-04-08',
        (SELECT EMP_SAL
         FROM EMP
         WHERE EMP_NO = 2555),
        NULL, NULL ),
    ROW (2446, (SELECT EMP_NAME
                FROM EMP
                WHERE EMP_NO = 2556),
        '1978-05-09',
        (SELECT EMP_SAL
         FROM EMP
         WHERE EMP_NO = 2556),
        NULL, NULL );
```



После выполнения этой операции в таблице EMP появятся две новые строки для служащих с уникальными идентификаторами 2445 и 2446, причем первому из них будет присвоено имя и размер заработной платы служащего с уникальным идентификатором 2555, а второму – аналогичные данные о служащем с уникальным идентификатором 2556.

ВСТАВКА СТРОК РЕЗУЛЬТАТА ЗАПРОСА



Это вариант оператора вставки, когда набор вставляемых строк определяется через спецификацию запроса.

ПРИМЕР Требуется сохранить в отдельной таблице DEPT_SUMMARY сведения о числе служащих каждого отдела, их максимальной, минимальной и суммарной заработной плате.

DEPT_SUMMARY:

DEPT_NO : DEPT_NO
DEPT_EMP_NO : INTEGER
DEPT_MAX_SAL : SALARY
DEPT_MIN_SAL : SALARY
DEPT_TOTAL_SAL : SALARY

```
INSERT INTO DEPT_SUMMARY
  (SELECT DEPT_NO, COUNT(*), MAX (EMP_SAL),
    MIN (EMP_SAL), SUM (EMP_SAL)
   FROM EMP
  GROUP BY DEPT_NO);
```

ОПЕРАТОР UPDATE



Общий синтаксис оператора UPDATE выглядит следующим образом:

```
UPDATE table_name SET update_assignment_commalist
WHERE conditional_expression
update_assignment ::= column_name =
{ value_expression | DEFAULT | NULL }
```

Как это работает:

1. Для всех строк таблицы с именем `table_name` вычисляется булевское выражение `conditional_expression`. Строки, для которых значением этого булевского выражения является `true`, считаются подлежащими модификации (обозначим множество таких строк через T_m);
2. Каждая строка s (из T_m) подвергается модификации таким образом, что значение каждого столбца этой строки, указанного в списке `update_assignment_commalist`, заменяется значением, указанным в правой части соответствующего элемента списка модификации. Значения столбцов строки s , не указанные в списке модификации, остаются неизменными.

ПРИМЕР

Для всех служащих, работающих в отделах, заработная плата менеджеров которых превышает 30000 руб., установить размер заработной платы, на 1000 руб. превышающий средний размер заработной платы соответствующего отдела, а номера проектов, в которых участвуют эти служащие, сделать неопределенными.

```
UPDATE EMP SET EMP_SAL = (SELECT AVG (EMP1_SAL)
FROM EMP EMP1
WHERE EMP.DEPT_NO = EMP1.DEPT_NO)
+ 1000.00, PRO_NO = NULL
WHERE (SELECT EMP1.EMP_SAL
FROM EMP EMP1, DEPT
WHERE EMP.DEPT_NO = DEPT.DEPT_NO
AND DEPT_MNG = EMP1.EMP_NO AND) > 30000.00;
```

ОПЕРАТОР DELETE



Общий синтаксис оператора DELETE выглядит следующим образом:

```
DELETE FROM table_name  
WHERE conditional_expression
```

В некотором смысле оператор DELETE является частным случаем оператора UPDATE (или, наоборот, действие оператора UPDATE представляет собой комбинацию действий операторов DELETE и INSERT).

Как это работает:

1. Для всех строк таблицы с именем `table_name` вычисляется булевское выражение `conditional_expression`. Строки, для которых значением этого булевского выражения является `true`, считаются подлежащими удалению (обозначим множество таких строк через T_d);
2. Каждая строка s (из T_d) удаляется из указанной таблицы.

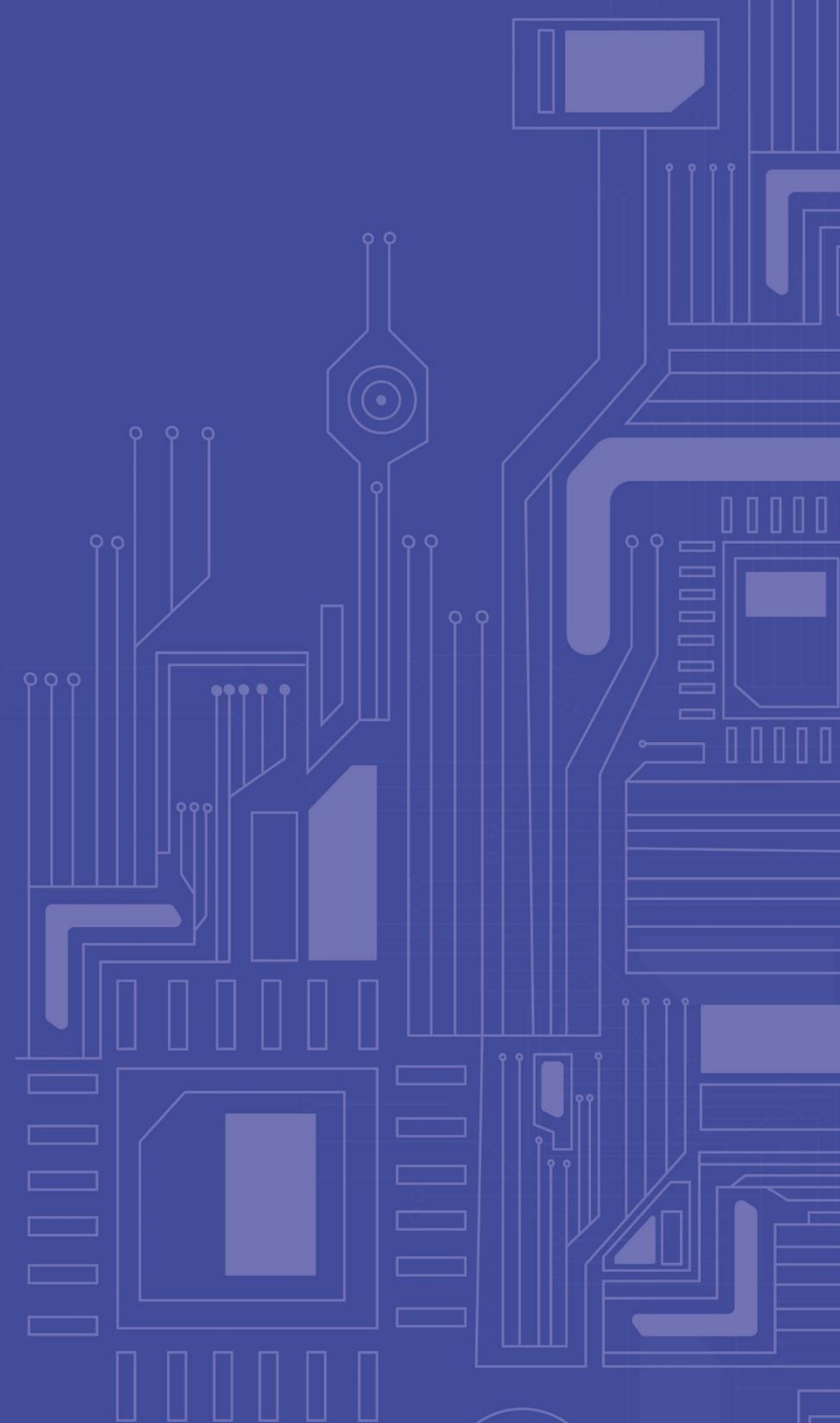
ПРИМЕР 1 Удалить из таблицы EMP все строки, относящиеся к служащим, которые участвуют в проекте с номером 772.

```
DELETE FROM EMP WHERE PRO_NO = 772;
```

ПРИМЕР 2 Удалить из таблицы EMP все строки, относящиеся к служащим, размер заработной платы которых превышает размер заработной платы менеджеров их отделов.

```
DELETE FROM EMP WHERE EMP_SAL >  
(SELECT EMP1.EMP_SAL  
FROM EMP EMP1, DEPT  
WHERE EMP.DEPT_NO = DEPT.DEPT_NO  
AND DEPT.DEPT_MNG = EMP1.EMP_NO);
```

РЕКУРСИВНЫЕ ЗАПРОСЫ



Обход дерева в ширину. При этом способе обхода непосредственные потомки обходятся слева направо, до того как производится переход к потомкам следующего уровня родства.

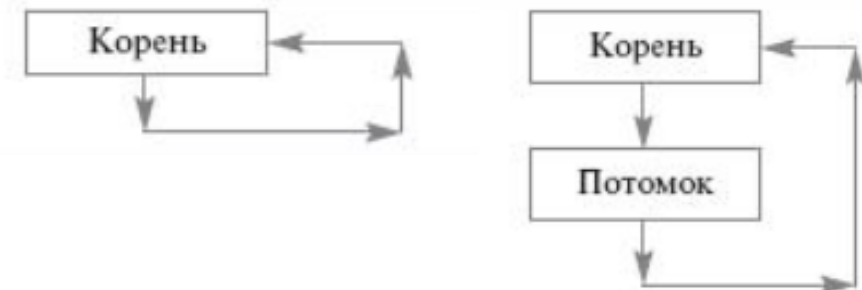
При обходе в ширину дерева узлы будут обходиться в следующем порядке:

Корень-Потомок1-Потомок2-Потомок3-П1.1-П1.2-П1.3-П2.1-П2.2-П2.3-П3.1-П3.2-П3.3.



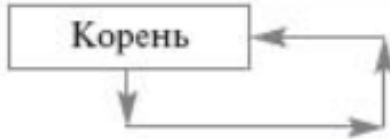
Обход дерева в глубину. При этом способе обхода на каждом шаге производится переход к самому левому текущему потомку. При обходе в глубину дерева порядок обхода узлов будет следующим: Корень-Потомок1-П1.1-П1.2-П1.3-Потомок2-П2.1-П2.2-П2.3-Потомок3-П3.1-П3.2-П3.3.

Цикл в ориентированном графе. В теории графов ориентированный граф называется циклическим в том и только в том случае, когда хотя бы один узел графа одновременно является и предком, и потомком (т. е. для этого узла имеется и выходящая, и входящая дуги).



Начиная с SQL:1999 узлами графа рекурсии являются строки, входящие в результат рекурсивного запроса, а дуги соответствуют способам обработки текущих строк, которые ведут к добавлению к результату новых строк.

Прямая рекурсия. По определению, некоторый элемент использует прямую рекурсию в том и только в том случае, когда он обращается сам к себе без посредников.



Пример не прямой рекурсии в графовой форме.



Линейная рекурсия. При линейно рекурсивном вызове элемент прямо рекурсивно обращается сам к себе не более одного раза. В SQL:1999 в определении любой виртуальной таблицы с рекурсией допускается не более одной ссылки на саму себя (в разделе FROM и/или в подзапросах).

Графовый пример рекурсии, не являющейся линейной.



МОНОТОННОСТЬ, ВЗАИМНАЯ РЕКУРСИЯ, ОТРИЦАНИЕ

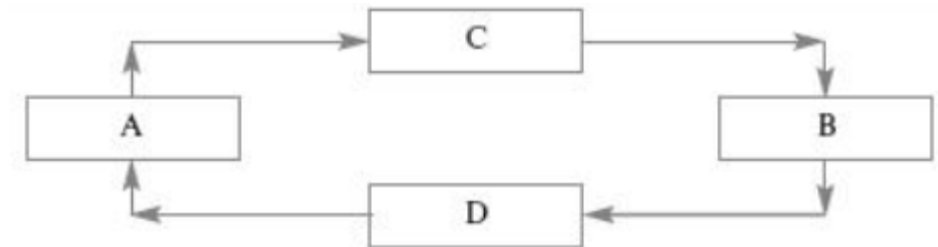


Монотонность. *Монотонной прогрессией* называется последовательность неубывающих или невозрастающих значений. Например, последовательность натуральных чисел $\{1, 2, \dots, n, \dots\}$ является монотонной. В SQL:1999 свойство монотонности поддерживается в том смысле, что число строк результата рекурсивного запроса не уменьшается на каждом шаге рекурсии.

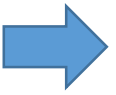
Взаимная рекурсия. Элементы A и B связаны отношением взаимной рекурсии, если A прямо или косвенно вызывает B, и B прямо или косвенно вызывает A.

Отрицание. В контексте SQL отрицанием называется любое действие, приводящее к уменьшению числа строк в результате запроса. Свойствами отрицания обладают операции над (мульти)множествами EXCEPT и INTERSECT, спецификация DISTINCT, условие NOT EXISTS и т.д.

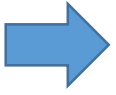
Графовый пример взаимной рекурсии (элемент A вызывает элемент B через элемент C, а элемент B вызывает элемент A через элемент D).



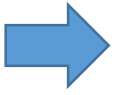
В стандарте SQL не запрещается использование отрицания в рекурсивных запросах. Возможной проблемы нарушения монотонности удастся избежать за счет того, что отрицание разрешается применять только к тем таблицам, которые являются полностью известными (или вычисленными) к моменту применения отрицания. В процессе вычисления таблицы применение к ней отрицания не допускается.



Начальный источник рекурсии. При выполнении рекурсивных вычислений обычно (хотя и не всегда) имеется некоторое начальное значение. В SQL этим начальным источником рекурсии является одна или несколько строк, удовлетворяющих некоторым начальным условиям. На основе этих строк в процессе рекурсивного вычисления производятся дополнительные строки, образующие окончательный результат.



Стратификация. В SQL рекурсивный запрос обычно состоит из «рекурсивной» и «нерекурсивной» частей. В процессе стратификации («расслоения») запроса выполнение этих двух частей разделяется. В более сложных рекурсивных запросах может содержаться несколько рекурсивных частей и более одной нерекурсивной части. В этом случае в процессе стратификации будет обнаружено большее число слоев.



Семантика фиксированной точки. В контексте SQL:1999 семантика фиксированной точки означает, что решение о завершении рекурсивного запроса принимается тогда, когда становится невозможно добавить к результату какие-либо дополнительные строки.

СИНТАКСИС РЕКУРСИИ С РАЗДЕЛОМ WITH



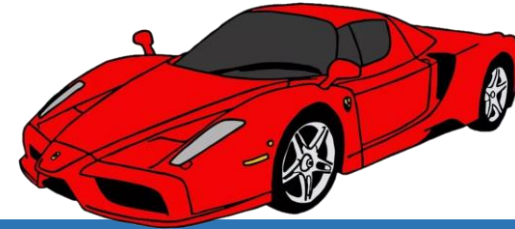
Полный синтаксис раздела WITH выглядит следующим образом:

```
with_clause ::= WITH [ RECURSIVE ]
with_element_comma_list
with_element ::= query_name [ (column_name_list) ]
               AS ( query_expression ) [ search_or_cycle_clause ]
search_or_cycle_clause ::= search_clause
                        | cycle_clause
                        | search_clause cycle_clause
search_clause ::= SEARCH recursive_search_order SET
               sequence_column_name
recursive_search_order ::= DEPTH FIRST BY
                        order_item_comma_list
                        | BREATH FIRST BY order_item_comma_list
cycle_clause ::= CYCLE cycle_column_name_comma_list
               SET cycle_mark_column_name TO value_expression
               DEFAULT value_expression
               USING path_column_name
```

КЛАССИЧЕСКИЙ ПРИМЕР РЕКУРСИИ (1/2)



Для иллюстрации возможностей рекурсивных запросов с разделом WITH и пояснения смысла конструкций SEARCH и CYCLE воспользуемся классическим примером «разборки деталей» (в данном случае мы будем разбирать автомобиль). Предположим, что данные о конструктивных элементах автомобиля хранятся в таблице CAR, определенной следующим образом:



```
CREATE TABLE CAR (CONTAINING_PART VARCHAR (10),  
CONTAINED_PART VARCHAR (10),  
NUMBER_OF_PARTS INTEGER,  
PART_COST DECIMAL (6,2));
```

У автомобиля имеется один конструктивный элемент верхнего уровня – полностью собранный автомобиль. Этот элемент не является составной частью какого-либо другого элемента, и для его строки значением столбца CONTAINING_PART является текстовая строка длины 0. В любой другой строке таблицы CAR, соответствующей некоторому неатомарному конструктивному элементу **e**, столбец CONTAINING_PART содержит идентификационный номер элемента **e1**, в который входит элемент **e**, столбец NUMBER_OF_PARTS – число экземпляров элемента **e**, входящих в **e1**, а столбец CONTAINED_PART – идентификационный номер самого элемента **e**. В любой строке таблицы CAR, соответствующей некоторому атомарному конструктивному элементу, значением столбца CONTAINED_PART является строка длины 0, а в столбце PART_COST сохраняется цена атомарного конструктивного элемента (для неатомарных элементов значение этого столбца равно нулю).

КЛАССИЧЕСКИЙ ПРИМЕР РЕКУРСИИ (2/2)



ЗАДАЧА

Требуется разобрать автомобиль, начиная с элемента самого верхнего уровня, и для каждого конструктивного элемента получить его номер, общее число используемых экземпляров этого элемента, а также, если элемент является атомарным, общую стоимость используемых экземпляров.

```
WITH RECURSIVE PARTS (PART_NUMBER,  
    NUMBER_OF_PARTS, COST) AS  
    (SELECT CONTAINED_PART, 1, 0.00 (a)  
    FROM CAR  
    WHERE CONTAINING_PART = ''  
    UNION ALL  
    SELECT CAR.CONTAINED_PART, CAR.NUMBER_OF_PARTS,  
        CAR.NUMBER_OF_PARTS * CAR.PART_COST  
    FROM CAR, PARTS  
    WHERE PARTS.PART_NUMBER = CAR.CONTAINING_PART)  
SELECT PART_NUMBER, SUM(NUMBER_OF PARTS),  
    SUM(COST) (b)  
FROM PARTS  
GROUP BY PART_NUMBER;
```

При вычислении раздела FROM основного запроса (b) начнется выполнение рекурсивного выражения запросов (a), определенного в разделе WITH. На первом шаге рекурсии будет выполнена часть данного выражения, предшествующая операции UNION ALL и образующая начальный источник рекурсии. В результате будет произведено исходное состояние виртуальной таблицы PARTS, в котором, в нашем случае, появится единственная строка, соответствующая автомобилю целиком. На следующем шаге к таблице PARTS будут добавлены строки, соответствующие конструктивным элементам второго уровня (для автомобиля это, по-видимому, двигатель, колеса, шасси и т.д.).

Этот процесс будет продолжаться до тех пор, пока мы не дойдем до атомарных конструктивных элементов и не достигнем, тем самым, фиксированной точки. Поскольку в рекурсивном запросе содержится операция UNION ALL, в результирующей таблице могут появляться строки-дубликаты. Наличие строки-дубликата вида <part_no, number, cost> означает, что элемент с номером part_no входит в одном и том же числе экземпляров в несколько конструктивных элементов более высокого уровня.

В предыдущем примере не определялся порядок, в котором строки добавляются к частичному результату рекурсивного запроса. иногда требуется, чтобы иерархия обходилась в глубину или в ширину. Соответствующая возможность обеспечивается конструкцией SEARCH. При указании требования обхода в глубину гарантируется, что каждый элемент-предок появится в результате раньше своих потомков и своих братьев справа. Если указывается требование обхода иерархии в ширину, в результате все братья одного уровня появляются раньше, чем какой-либо их потомок.

В списке столбцов сортировки раздела SEARCH должны указываться имена столбцов виртуальной таблицы, определенной в разделе WITH. Поскольку в данном случае мы хотим, чтобы в результате сначала появлялись все конструктивные элементы одного уровня (CONTAINING_PART), а затем все их подэлементы (CONTAINED_PART), в список выборки рекурсивного запроса PARTS добавлен столбец CONTAINING_PART, который не используется нигде, кроме раздела SEARCH. В разделе SET к результирующей таблице рекурсивного запроса добавлен столбец, который мы назвали ORDER_COLUMN. Название соответствует природе столбца, потому что при выполнении рекурсивного запроса в этот столбец автоматически заносятся значения, характеризующие порядок генерируемых строк в соответствии с выбранным способом обхода иерархии. Чтобы строки результата основного запроса появлялись в должном порядке, в этом запросе требуется наличие раздела ORDER BY с указанием столбца, определенного в разделе SET.

Вариант запроса, в котором содержится раздел SEARCH с требованием обхода иерархии элементов автомобиля в ширину.

```
WITH RECURSIVE PARTS (ASSEMBLY, PART_NUMBER,
    NUMBER_OF_PARTS, COST) AS
    (SELECT CONTAINING_PART, CONTAINED_PART, 1, 0.00
    FROM CAR
    WHERE CONTAINING_PART = ''
    UNION ALL
    SELECT CAR.CONTAINING_PART, CAR.CONTAINED_PART,
    CAR.NUMBER_OF_PARTS, CAR.NUMBER_OF_PARTS *
    CAR.PART_COST
    FROM CAR, PARTS
    WHERE PARTS.PART_NUMBER = CAR.CONTAINING_PART)
SEARCH BREADTH FIRST
BY CONTAINING_PART, CONTAINED_PART
SET ORDER_COLUMN
SELECT PART_NUMBER, NUMBER_OF PARTS, COST
FROM PARTS
ORDER BY ORDER_COLUMN;
```

Иногда сами данные, хранимые в таблицах базы данных, могут иметь циклическую природу.

ПРИМЕР

Представим себе, например, компанию, в которой существует совет директоров, являющийся высшим органом управления компанией. Обычным случаем является тот, когда по крайней мере один из членов совета директоров является простым служащим этой же компании (например, он может входить в совет директоров как представитель профсоюза). Назовем данного члена совета директоров EMP_DIR. Как член совета директоров, EMP_DIR «управляет» деятельностью президента компании. С другой стороны, как служащий компании, EMP_DIR находится в прямом или косвенном подчинении у президента компании. Такое положение может привести к зацикливанию выполнения рекурсивных запросов.

Раздел CYCLE обеспечивает некоторую возможность распознавать подобные ситуации. Если у пользователя имеется полная уверенность в отсутствии циклов в данных, к которым адресуется рекурсивный запрос, то использование раздела CYCLE не требуется.

➡ Подход к распознаванию зацикленных запросов, принятый в SQL, состоит в том, что распознаются данные, которые уже участвовали ранее в формировании результата рекурсивного запроса. При наличии раздела CYCLE при добавлении к результату строк, удовлетворяющих условию запроса, такие строки помечаются указанным значением, которое означает, что эти строки уже вошли в результат. При попытке добавления к результату каждой новой строки проверяется, не находится ли она уже в результате, т. е. не помечена ли она этим указанным в разделе CYCLE значением. Если это действительно так, то считается, что имеет место цикл, и дальнейшее выполнение рекурсивного запроса прекращается.

Синтаксис раздела CYCLE.

```
cycle_clause ::= CYCLE cycle_column_name_comma_list  
               SET cycle_mark_column_name TO value_expression_1  
               DEFAULT value_expression_2  
               USING path_column_name
```

В списке `cycle_column_name_comma_list` указываются имена одного или нескольких столбцов, которые используются для идентификации новых строк результата на основе строк, уже входящих в результат. Например, в примере столбец `CONTAINED_PART` связывает конструктивный элемент автомобиля с входящими в его состав подэлементами (через значения их столбцов `CONTAINING_PART`).

Раздел `SET` приводит к образованию нового столбца результирующей таблицы. Для строк, которые попадают в результат первый раз, в столбец `cycle_mark_column_name` заносится значение выражения `value_expression_2`. В повторно заносимых строках значение столбца – `value_expression_1`. Типом данных этого столбца является тип символьных строк длины один, так что в качестве `value_expression_1` и `value_expression_2` разумно использовать константы '0' и '1' или 'Y' и 'N'.

Раздел `USING` приводит к образованию еще одного дополнительного столбца результата с именем `path_column_name`. Типом данных столбца является `ARRAY`, причем кардинальность этого типа предполагается достаточно большой, чтобы сохранить информацию обо всех строках, попавших в результат. Элементы массива имеют «строчный тип» (`row type`), содержащий столько столбцов, сколько их указано в списке раздела `CYCLE`. Каждый элемент массива соответствует строке результата, и в его столбцах содержится копия значений соответствующих столбцов этой строки.

```
WITH RECURSIVE PARTS (PART_NUMBER,  
    NUMBER_OF_PARTS, COST) AS  
    (SELECT CONTAINED_PART, 1, 0.00  
     FROM CAR  
     WHERE CONTAINING_PART = ''  
    UNION ALL  
     SELECT CAR.CONTAINED_PART, CAR.NUMBER_OF_PARTS,  
           CAR.NUMBER_OF_PARTS * CAR.PART_COST  
     FROM CAR, PARTS  
     WHERE PARTS.PART_NUMBER = CAR.CONTAINING_PART)  
    CYRCLE CONTAINED_PART  
    SET CYCLEMARK TO 'Y' DEFAULT 'N'  
    USING CYRCLEPATH  
SELECT PART_NUMBER, SUM(NUMBER_OF PARTS), SUM(COST)  
FROM PARTS  
ORDER BY PART_NUMBER;
```

Имена столбцов CYCLEMARK и CYCLEPATH выбраны произвольным образом – требуется только, чтобы имена этих столбцов отличались от имен столбцов рекурсивного запроса. При выполнении запроса строки, удовлетворяющие его условию, накапливаются в результирующей таблице. Но, кроме того, эти строки «кэшируются» в столбце CYCLEPATH. При попытке добавления к результату новой строки на основе текущего содержимого столбца CYCLEPATH проверяется, не содержится ли она уже в результате.

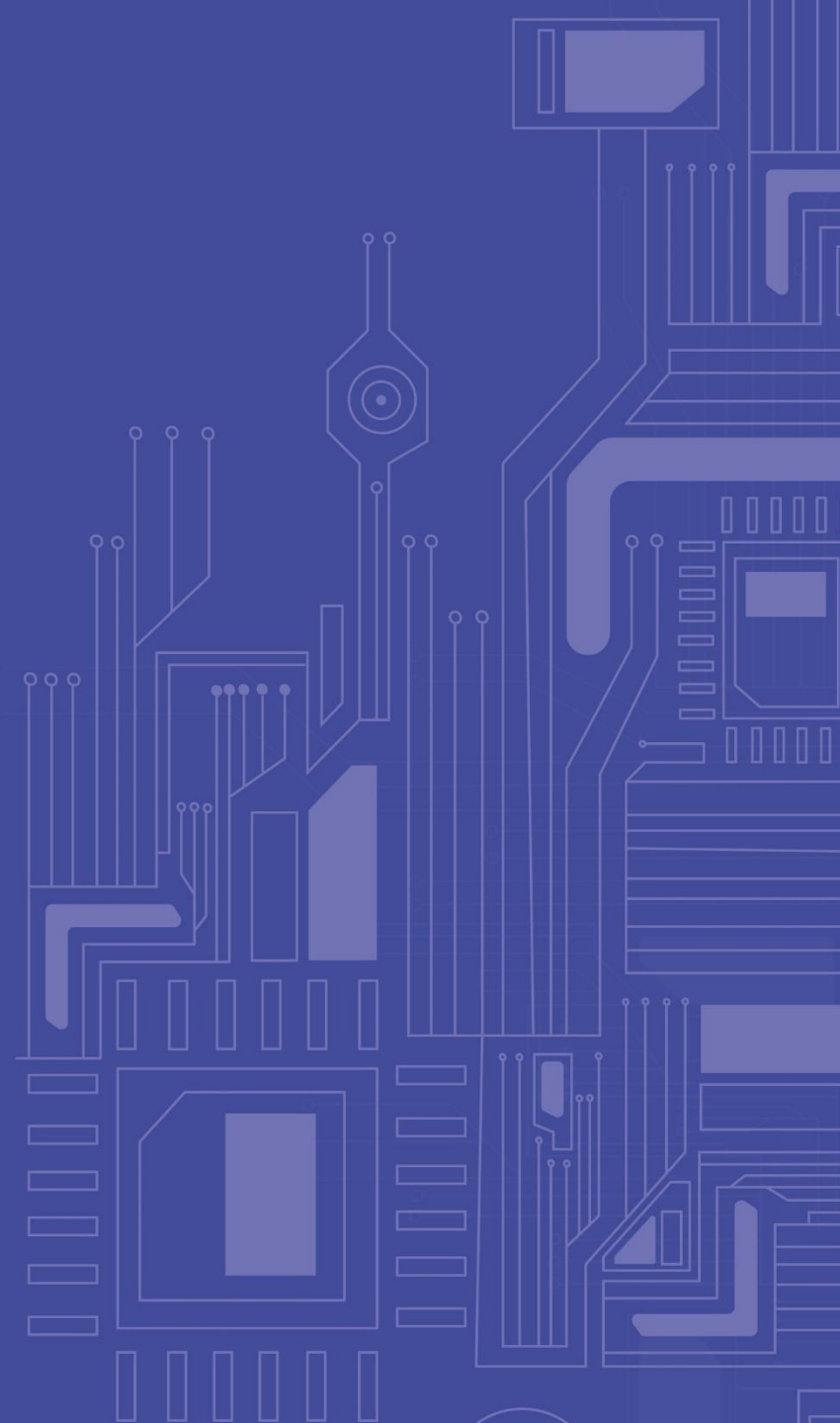
Если не содержится, то данные об этой строке добавляются к столбцу CYCLEPATH (к массиву добавляется новый элемент), в столбец CYCLEMARK этой строки заносится значение 'N', и строка добавляется к результату. Иначе в столбец CYCLEMARK соответствующей строки результата заносится значение 'Y', означающее, что от этой строки начинается цикл.

Рекурсивным называется *представление*, в определяющем выражении запроса которого используется имя этого же представления. В представлениях может использоваться и прямая, и взаимная рекурсия. Синтаксис оператора определения рекурсивного запроса выглядит следующим образом:

```
CREATE RECURSIVE VIEW table_name  
  [ column_name_comma_list ]  
  AS query_expression
```

Хотя для того, чтобы представление было рекурсивным, требуется рекурсивность определяющего выражения запроса (т.е. в нем должна присутствовать спецификация RECURSIVE); наличие избыточного ключевого RECURSIVE в определении рекурсивного представления является обязательным. Как говорят авторы стандарта, это сделано для того, чтобы избежать случайного появления непредусмотренных рекурсивных представлений.

ОКОННЫЕ ФУНКЦИИ



КАКИЕ ЗАДАЧИ МОЖНО РЕШАТЬ:

- **Ранжирование** (всевозможные рейтинги)
- **Сравнение со смещением** (соседние элементы и интервалы)
- **Агрегация** (сумма, среднее, количество элементов)
- **Скользящие агрегаты** (сумма и среднее в динамике)



employees

id	name	department	salary
11	Мария	hr	70
12	Светлана	hr	78
21	Сергей	it	84
22	Владимир	it	90
23	Надежда	it	104
24	Георгий	it	104
25	Татьяна	it	120
31	Владислав	sales	96
32	Ольга	sales	96
33	Андрей	sales	100



rank	name	department	salary
1	Татьяна	it	120
2	Надежда	it	104
2	Георгий	it	104
3	Андрей	sales	100
4	Владислав	sales	96
4	Ольга	sales	96
5	Владимир	it	90
6	Сергей	it	84
7	Светлана	hr	78
8	Мария	hr	70

```
select  
  dense_rank() over w as "rank",  
  name, department, salary  
from employees  
window w as (order by salary desc)  
order by "rank", id;
```

window – ключевое слово,
которое показывает, что дальше
будет определение окна

`dense_rank()` — это оконная функция, которая считает ранг по указанному окну. Логика `dense_rank()` — начать с 1 и увеличивать ранг каждый раз, когда очередное значение из окна отличается от предыдущего.

Конструкция **window** сама по себе ничего не делает с результатами запроса. Она только определяет окно, которое можно использовать (или не использовать) в запросе. Если убрать вызов `dense_rank()`, запрос отработает, как будто нет никаких окон.

Ни Oracle, ни SQL server не поддерживают конструкцию **window**. Чтобы выполнялся оконный запрос в этих СУБД, нужно перенести описание окна внутрь инструкции **over**.

```
select
  dense_rank() over w as "rank",
  name, department, salary
from employees
window w as (order by salary desc)
order by "rank", id;
```



```
select
  dense_rank() over (
    order by salary desc
  ) as "rank",
  name, department, salary
from employees
order by "rank", id;
```

```
select
  dense_rank() over w1 as r_asc,
  dense_rank() over w1 as r_desc,
  name, salary
from employees
window
  w1 as (order by salary asc),
  w2 as (order by salary desc)
order by salary, id;
```



r_asc	r_desc	name	salary
1	8	Мария	70
2	7	Светлана	78
3	6	Сергей	84
4	5	Владимир	90
5	4	Владислав	96
5	4	Ольга	96
6	3	Андрей	100
7	2	Надежда	104
7	2	Георгий	104
8	1	Татьяна	120

```
select
  dense_rank() over w as "rank",
  name, department, salary
from employees
window w as (
  partition by department
  order by salary desc
)
order by "rank", id;
```

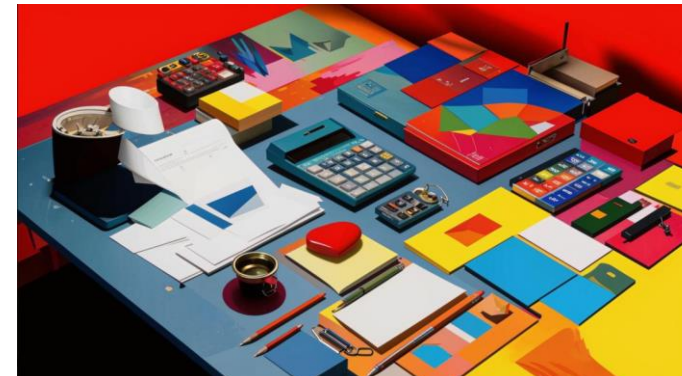


rank	name	department	salary
1	Светлана	hr	78
2	Мария	hr	70
1	Татьяна	it	120
2	Надежда	it	104
2	Георгий	it	104
3	Владимир	it	90
4	Сергей	it	84
1	Андрей	sales	100
2	Владислав	sales	96
2	Ольга	sales	96

1

```
select
  ntile(3) over w as tile,
  name, department, salary
from employees
window w as (order by salary desc)
order by salary desc, id;
```

tile	name	department	salary
1	Татьяна	it	120
1	Надежда	it	104
1	Георгий	it	104
1	Андрей	sales	100
2	Владислав	sales	96
2	Ольга	sales	96
2	Владимир	it	90
3	Сергей	it	84
3	Светлана	hr	78
3	Мария	hr	70



2

```
select
  ntile(2) over w as tile,
  name, department, salary
from employees
window w as (order by salary desc, id)
order by salary desc, id;
```

ntile(n) – разбивает записи на n равных групп. Первые группы больше при неравных размерах групп. Делит по количеству, не по смежным значениям!

tile	name	department	salary
1	Татьяна	it	120
1	Надежда	it	104
1	Георгий	it	104
1	Андрей	sales	100
1	Владислав	sales	96
2	Ольга	sales	96
2	Владимир	it	90
2	Сергей	it	84
2	Светлана	hr	78
2	Мария	hr	70

ФУНКЦИИ РАНЖИРОВАНИЯ

Функция	Описание
<code>row_number()</code>	Порядковый номер строки
<code>dense_rank()</code>	Ранг строки
<code>rank()</code>	Ранг строки, но с пропусками
<code>ntile(n)</code>	Разбивает все строки на n групп, и возвращает номер группы, куда попала строка



`dense_rank()`



rank	name	department	salary
1	Татьяна	it	120
2	Надежда	it	104
2	Георгий	it	104
3	Андрей	sales	100
4	Владислав	sales	96
4	Ольга	sales	96
5	Владимир	it	90
6	Сергей	it	84
7	Светлана	hr	78
8	Мария	hr	70



`rank()`



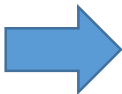
rank	name	department	salary
1	Татьяна	it	120
2	Надежда	it	104
2	Георгий	it	104
4	Андрей	sales	100
5	Владислав	sales	96
5	Ольга	sales	96
7	Владимир	it	90
8	Сергей	it	84
9	Светлана	hr	78
10	Мария	hr	70



СРАВНЕНИЕ СО СМЕЩЕНИЕМ

Сравнение со смещением – это изучение разницы между соседними элементами, или сравнение элементов набора с границами набора. Например, насколько отличаются 7е и 8е место рейтинга, или насколько отличается 3е место от первого.

id	name	department	salary
11	Мария	hr	70
12	Светлана	hr	78
21	Сергей	it	84
22	Владимир	it	90
23	Надежда	it	104
24	Георгий	it	104
25	Татьяна	it	120
31	Владислав	sales	96
32	Ольга	sales	96
33	Андрей	sales	100



name	department	salary	diff
Мария	hr	70	
Светлана	hr	78	11%
Сергей	it	84	8%
Владимир	it	90	7%
Владислав	sales	96	7%
Ольга	sales	96	0%
Андрей	sales	100	4%
Надежда	it	104	4%
Георгий	it	104	0%
Татьяна	it	120	15%

```

select
  name, department, salary,
  round(
    (salary – lag(salary, 1) over w)*100
    / lag(salary, 1) over w
  ) as diff
from employees
window w as (order by salary, id)
order by salary, id;

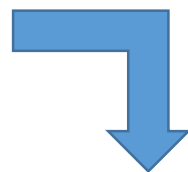
```

Функция **lag()** возвращает значение из указанного столбца, отстоящее от текущего на указанное количество записей назад. В нашем случае – предыдущую запись столбца *salary*.

СРАВНЕНИЕ С ГРАНИЦАМИ



id	name	department	salary
11	Мария	hr	70
12	Светлана	hr	78
21	Сергей	it	84
22	Владимир	it	90
23	Надежда	it	104
24	Георгий	it	104
25	Татьяна	it	120
31	Владислав	sales	96
32	Ольга	sales	96
33	Андрей	sales	100



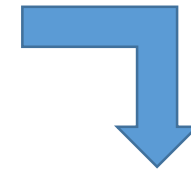
id	name	department	salary	low	high
11	Мария	hr	70	70	70
12	Светлана	hr	78	70	78
21	Сергей	it	84	84	84
22	Владимир	it	90	84	90
23	Надежда	it	104	84	104
24	Георгий	it	104	84	104
25	Татьяна	it	120	84	120
31	Владислав	sales	96	96	96
32	Ольга	sales	96	96	96
33	Андрей	sales	100	96	100

select

```
name, department, salary,  
first_value(salary) over w as low,  
last_value(salary) over w as high  
from employees  
window w as (  
    partition by department  
    order by salary  
)  
order by department, salary, id;
```

Функции `first_value()` и `last_value()` указанного столбца работают не с секциями, а с фреймами

Фрейм находится в той же секции, что и текущая запись.
 Начало фрейма = начало секции.
 Конец фрейма = последняя запись секции с текущим значением.



Иван	84	← начало фрейма
Марья	90	
Никодим	104	← текущая запись
Ульяна	104	← конец фрейма
Проход	120	

Иван	84	← начало фрейма
Марья	90	
Никодим	104	← текущая запись,
Ульяна	114	конец фрейма
Проход	120	

Иван	84	← начало фрейма
Марья	90	
Никодим	104	← текущая запись
Ульяна	104	
Проход	104	← конец фрейма

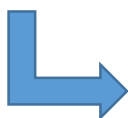
```
select
  name, department, salary,
  first_value(salary) over w as low,
  last_value(salary) over w as high
from employees
window w as (
  partition by department
  order by salary
  rows between unbounded preceding
  and unbounded following
)
order by department, salary, id;
```

name	department	salary	low	high
Марья	hr	70	70	78
ветлана	hr	78	70	78
ергей	it	84	84	120
ладимир	it	90	84	120
адежда	it	104	84	120
еоргий	it	104	84	120
атьяна	it	120	84	120
ладислав	sales	96	96	100
ольга	sales	96	96	100
ндрей	sales	100	96	100

Агрегация – это когда мы считаем суммарные или средние показатели (*агрегаты*) по группе данных.

ПРИМЕР: Хотим узнать, какой процент оплаты труда отдела уходит каждому сотруднику

id	name	department	salary
11	Мария	hr	70
12	Светлана	hr	78
21	Сергей	it	84
22	Владимир	it	90
23	Надежда	it	104
24	Георгий	it	104
25	Татьяна	it	120
31	Владислав	sales	96
32	Ольга	sales	96
33	Андрей	sales	100



name	department	salary	fund	perc
Мария	hr	70	148	47%
Светлана	hr	78	148	53%
Сергей	it	84	502	17%
Владимир	it	90	502	18%
Надежда	it	104	502	21%
Георгий	it	104	502	21%
Татьяна	it	120	502	24%
Владислав	sales	96	292	33%
Ольга	sales	96	292	33%
Андрей	sales	100	292	34%

```
select
  name, department, salary,
  sum(salary) over w as fund,
  round(salary*100 / sum(salary) over w)
as perc
from employees
window w as (partition by department)
order by department, salary, id;
```



Порядок строк внутри секции нам не важен, sum() считает по секции, не по фрейму

ФИЛЬТРАЦИЯ И ПОРЯДОК ВЫПОЛНЕНИЯ



id	name	city	department	salary
11	Мария	Обнинск	hr	70
12	Светлана	Обнинск	hr	78
21	Сергей	Обнинск	it	84
22	Владимир	Москва	it	90
23	Надежда	Обнинск	it	104
24	Георгий	Москва	it	104
25	Татьяна	Москва	it	120
31	Владислав	Москва	sales	96
32	Ольга	Обнинск	sales	96
33	Андрей	Москва	sales	100

ПРИМЕР: Хотим оставить в отчете только обнинских сотрудников.

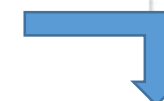
```
select
  name, salary,
  sum(salary) over w as fund,
from employees
where city = 'Обнинск'
window w as (partition by
department)
order by department, salary, id;
```



name	salary	fund
Мария	70	148
Светлана	78	148
Сергей	84	188
Надежда	104	188
Ольга	96	96

1. Взять таблицы (**from**) и при необходимости соединить (**join**).
2. Отфильтровать строки (**where**).
3. Сгруппировать строки (**group by**)
4. Отфильтровать (**having**)
5. Взять конкретные столбцы (**select**)
6. Рассчитать **function () over window**
7. Отсортировать (**order by**)

```
with emp as (
  select
    name, city, salary,
    sum(salary) over w as fund,
  from employees
  window w as (partition by department)
  order by department, salary, id
)
select name, salary, fund
from emp where city = 'Обнинск' ;
```



name	salary	fund
Мария	70	148
Светлана	78	148
Сергей	84	502
Надежда	104	502
Ольга	96	292



```
select
  name, salary,
  sum(salary) over w as fund,
from employees
where city = 'Обнинск'
window w as (partition by
  department)
order by department, salary, id;
```



Окно можно определять явно через **window** – это самый наглядный вариант

```
select
  name, salary,
  sum(salary) over (partition by
  department) as fund,
from employees
where city = 'Обнинск'
order by department, salary, id;
```



Окно можно определять просто как параметр **over** – это самый популярный вариант

```
select
  name, salary,
  sum(salary) over () as fund,
from employees
where city = 'Обнинск'
order by department, salary, id;
```



Окно можно не определять как пустой как параметр **over** – это тоже нормально

Скользящее среднее

year	month	income	expense
2024	1	94	82
2024	2	94	75
2024	3	94	104
2024	4	100	94
2024	5	100	99
2024	6	100	105
2024	7	100	95
2024	8	100	110
2024	9	104	104

Доходы растут, а
расходы?

А расходы растут!

Чтобы **сгладить скачки**, считаем среднее с учетом предыдущего и следующего месяца:

- скользящее среднее за февраль = (январь + февраль + март) / 3;
- за март = (февраль + март + апрель) / 3;
- за апрель = (март + апрель + май) / 3; + и так далее.

```
select
  year, month, expense,
  round(avg(expense) over w) as roll_avg
from expenses
where year = 2020 and month <= 9
window w as (
  order by year, month
  rows between 1 preceding and 1 following
)
order by year, month;
```



year	month	expense	roll_avg
2024	1	82	79
2024	2	75	87
2024	3	104	91
2024	4	94	99
2024	5	99	99
2024	6	105	100
2024	7	95	103
2024	8	110	103
2024	9	104	107



ФРЕЙМ (1/2)

В общем случае определение фрейма выглядит так:
rows between X preceding and Y following
где X – количество строк перед текущей, а Y – количество строк после текущей.

year	month	expense
2024	1	82
2024	2	75
2024	3	104
2024	4	94
2024	5	99
2024	6	105
2024	7	95
2024	8	110
2024	9	104

rows between
2 preceding
and
1 following

year	month	expense
2024	1	82
2024	2	75
2024	3	104
2024	4	94
2024	5	99
2024	6	105
2024	7	95
2024	8	110
2024	9	104

rows between
unbounded preceding
and
1 following

year	month	expense
2024	1	82
2024	2	75
2024	3	104
2024	4	94
2024	5	99
2024	6	105
2024	7	95
2024	8	110
2024	9	104

rows between
1 preceding
and
unbounded following

Если указать вместо X или Y значение **unbounded** – это значит «от/од границы секции».



ФРЕЙМ (2/2)

year	month	expense
2024	1	82
2024	2	75
2024	3	104
2024	4	94
2024	5	99
2024	6	105
2024	7	95
2024	8	110
2024	9	104

rows between

2 preceding

and current row

year	month	expense
2024	1	82
2024	2	75
2024	3	104
2024	4	94
2024	5	99
2024	6	105
2024	7	95
2024	8	110
2024	9	104

rows between

current row and
2 following

year	month	expense
2024	1	82
2024	2	75
2024	3	104
2024	4	94
2024	5	99
2024	6	105
2024	7	95
2024	8	110
2024	9	104

rows between

1 preceding

and

2 following

Если указать вместо **X preceding** или **Y following** значение **current row** – это значит «текущая запись». Фрейм никогда не выходит за границы секции, если столкнулся с ней – обрезается.



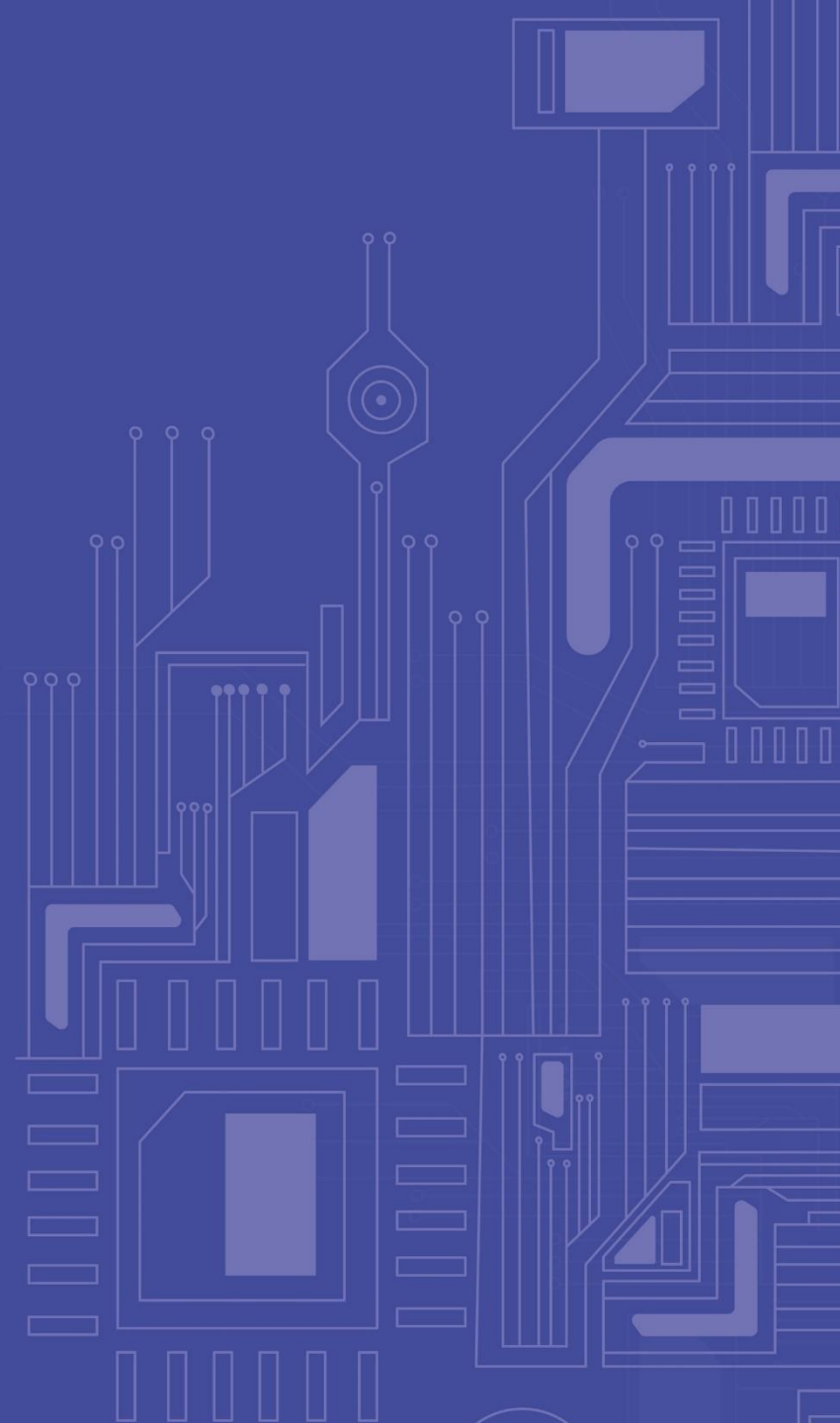
СУММА НАРАСТАЮЩИМ ИТОГОМ

Посчитаем доходы и расходы по месяцам нарастающим итогом.

month	income	expense	t_income	t_expense	t_profit
1	94	82	94	82	12
2	94	75	188	157	31
3	94	104	282	261	21
4	100	94	382	355	27
5	100	99	482	454	28
6	100	105	582	559	23
7	100	95	682	654	28
8	100	110	782	764	18
9	104	104	886	868	18

```
select
  month, income, expense,
  sum(income) over w as t_income,
  sum(expense) over w as t_expense,
  sum(income) over w - sum(expense) over w as t_profit
from expenses
where year = 2020 and month <= 9
window w as (
  order by year, month
  rows between unbounded preceding and current row
)
order by year, month;
```

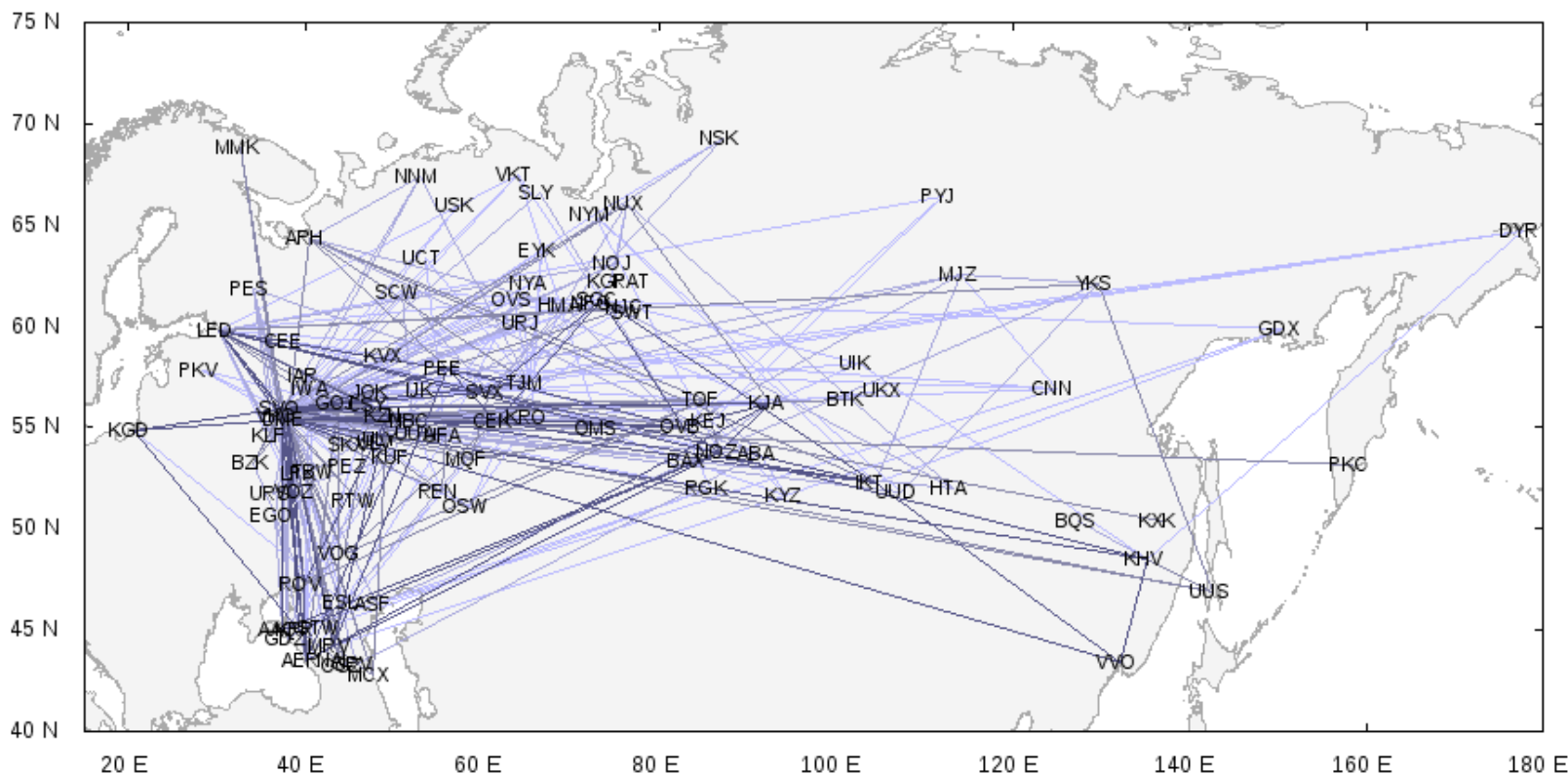
ПРИМЕР РЕКУРСИИ ИЗ НОВЕЙШЕЙ ПРАКТИКИ



<https://habr.com/ru/companies/postgrespro/articles/318398/> - статья с примером на рекурсию.

<https://habr.com/ru/companies/postgrespro/articles/316428/> - учебная база PostgreSQL.

База данных «Авиаперевозки»



<https://edu.postgrespro.ru/bookings.pdf>

- полное описание базы данных и инструкция по скачиванию.

- *Как реальное время полета отличается от запланированного?*
- *Обычно полеты с запада на восток длинные (вылетаем ночью, прилетаем утром следующего дня), а с востока на запад — короткие (прилетаем в тот же день почти в то же время). А что происходит в демо-базе?*
- *Как распределено время бронирования и время регистрации по отношению к дате и времени рейса?*
- *Сколько человек обычно входит в одно бронирование?*
- *Бывают ли пассажиры, летящие туда-обратно? Всегда ли маршрут «туда» совпадает с маршрутом «обратно»?*

Старый добрый реляционный SQL хорошо справляется с неупорядоченными множествами: для них и в самом языке, и «под капотом» СУБД имеется целый арсенал средств. Чего SQL не любит, так это когда его дергают в цикле строчка за строчкой, вместо того, чтобы выполнить задачу одним оператором. Оптимизатор опускает руки и отходит в сторону, и вы остаетесь один на один с производительностью.

Так вот рекурсивный запрос — это такой способ устроить цикл прямо внутри SQL. Не то, чтобы это было нужно очень часто, но иногда все-таки приходится. И тут начинаются сложности, поскольку рекурсивный запрос не очень похож ни на обычный запрос, ни тем более на обычный цикл.

Сначала выполняется нерекурсивная часть (1). Затем рекурсивная часть (2) выполняется до тех пор, пока она возвращает какие-либо строки. Рекурсивная часть называется так потому, что она может обращаться к результату выполнения предыдущей итерации, который доступен ей под именем `t`.

Попутно результат выполнения каждой итерации складывается в результирующую таблицу, которая будет доступна под тем же именем `t` после того, как весь запрос отработает (3). Если вместо `UNION ALL` написать `UNION`, то на каждой итерации будут устраняться дубликаты строк.

Общая схема рекурсивного запроса такова:

```
WITH RECURSIVE t AS (  
    нерекурсивная часть      (1)  
  
    UNION ALL  
  
    рекурсивная часть        (2)  
)  
  
SELECT * FROM t;              (3)
```

ПРОСТОЙ ПРИМЕР



```
demo=# WITH RECURSIVE t(n,factorial) AS (  
  
VALUES (0,1)  
  
UNION ALL  
  
SELECT t.n+1, t.factorial*(t.n+1) FROM t WHERE t.n < 5  
  
)  
  
SELECT * FROM t;
```



n		factorial
---	--	-----------

-----+-----		
-------------	--	--

0		1
---	--	---

1		1
---	--	---

2		2
---	--	---

3		6
---	--	---

4		24
---	--	----

5		120
---	--	-----

(6 строк)

ТОТ САМЫЙ ПРИМЕР С РЕКУРСИЕЙ



Поиск кратчайшего пути из Усть-Кута (UKX) в Нерюнгри (CNN)

```
WITH RECURSIVE p(last_arrival, destination, hops, flights, found) AS (
```

```
  SELECT a_from.airport_code,
```

```
         a_to.airport_code,
```

```
         ARRAY[a_from.airport_code],
```

```
         ARRAY[]::char(6)[],
```

```
         a_from.airport_code = a_to.airport_code
```

```
  FROM    airports a_from, airports a_to
```

```
  WHERE   a_from.airport_code = 'UKX'
```

```
  AND     a_to.airport_code = 'CNN'
```

```
  UNION ALL
```

```
    SELECT r.arrival_airport,
```

```
           p.destination,
```

```
           (p.hops || r.arrival_airport)::char(3)[],
```

```
           (p.flights || r.flight_no)::char(6)[],
```

```
           bool_or(r.arrival_airport = p.destination) OVER ()
```

```
  FROM    routes r, p
```

```
  WHERE   r.departure_airport = p.last_arrival
```

```
  AND     NOT r.arrival_airport = ANY(p.hops)
```

```
  AND     NOT p.found
```

```
)
```

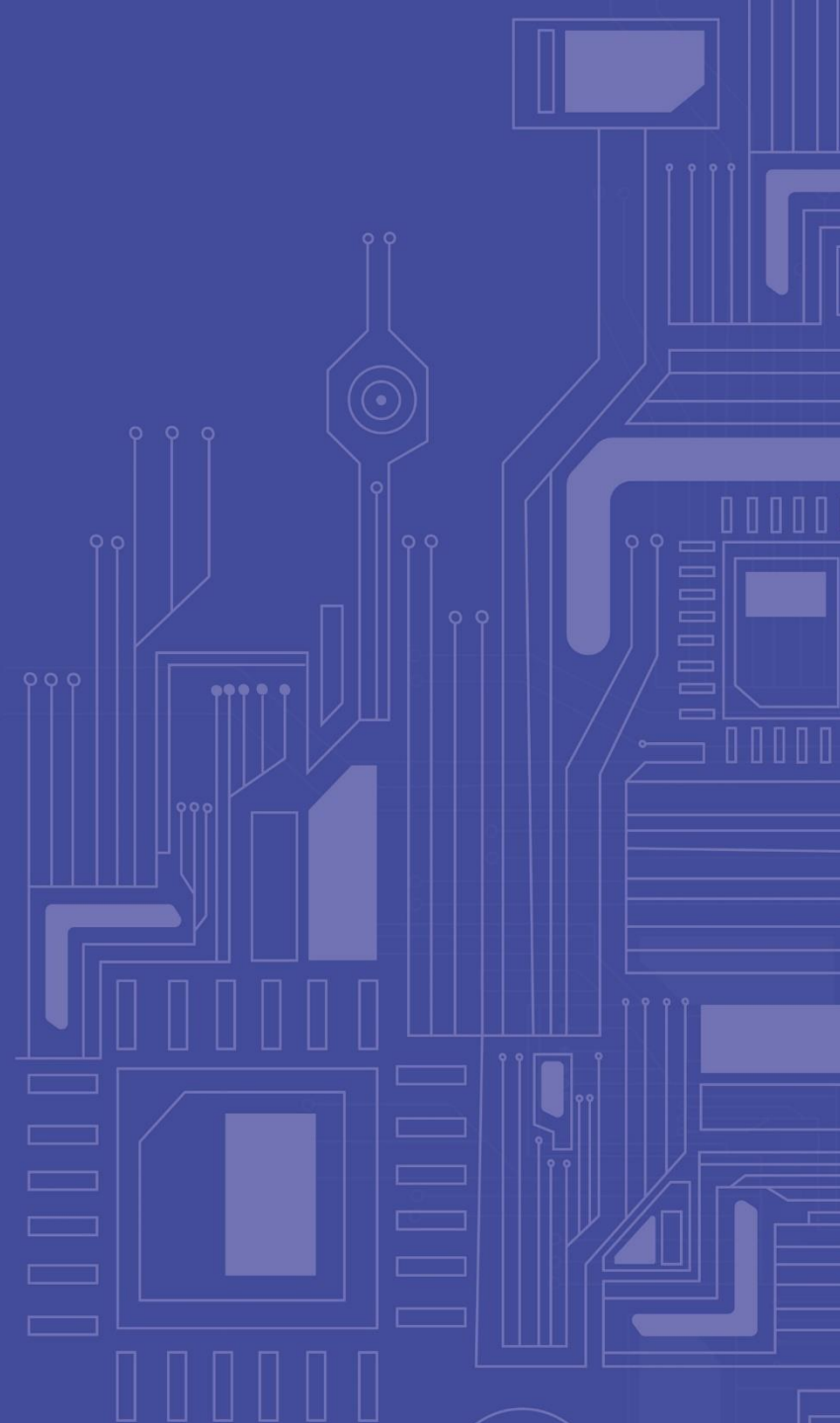
```
SELECT hops,
```

```
       flights
```

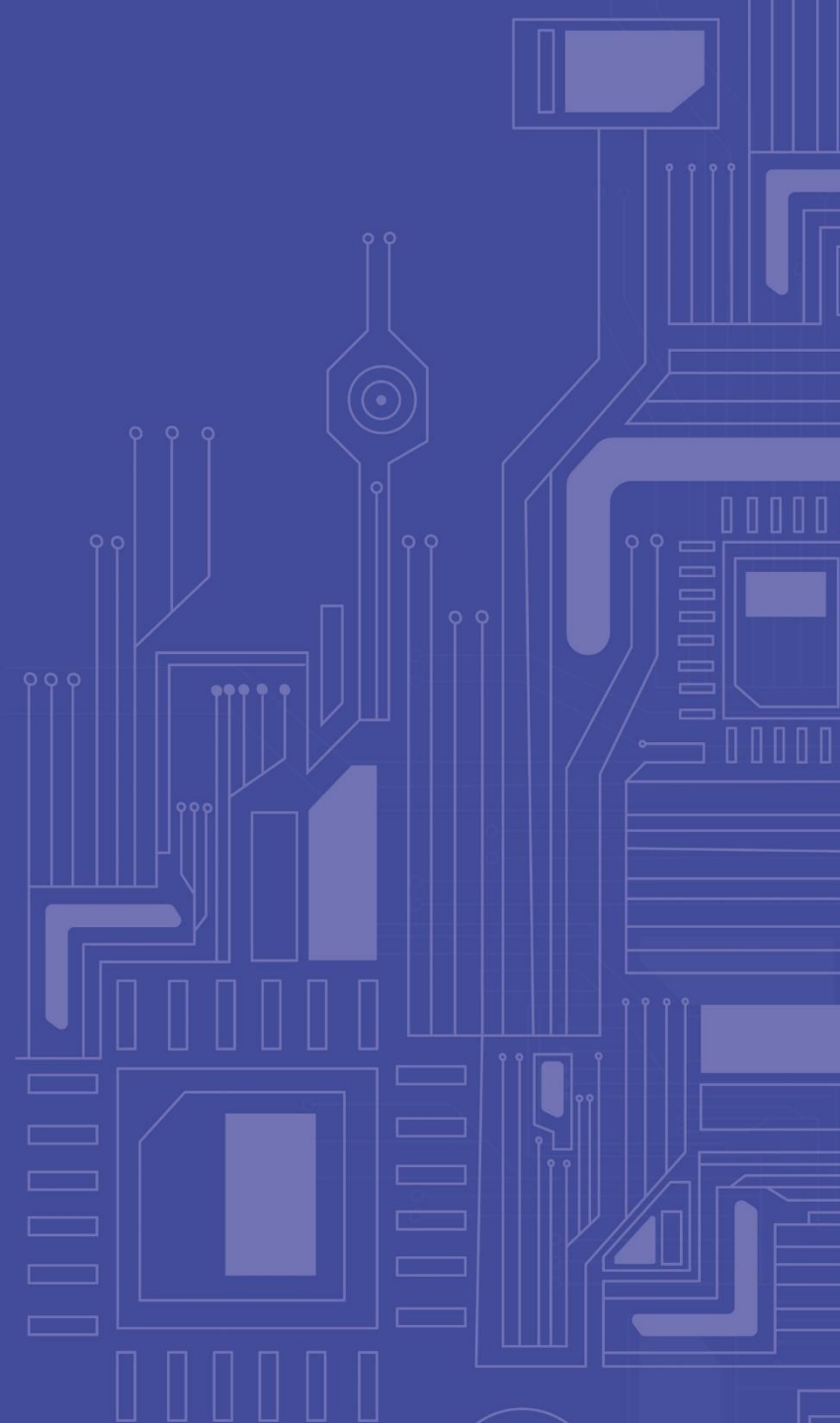
```
FROM   p
```

```
WHERE  p.last_arrival = p.destination;
```





СЕМИНАР



1. В группе лежит два файла .csv: users(1) и log(1). Надо скачать.
2. В этих файлах лежит информация о пользователях, делавших ставки, времени захода на платформу и выигрышах/проигрышах.
3. В log(1).csv есть данные идентификатора пользователя, времени посещения, дате посещения, размере ставки и размере выигрыша.
4. Данные из файла log(1).csv надо будет загрузить в таблицу LOG с колонками user_id, time, bet, win.
5. Файл users(1).csv в кодировке koï8_r. Перед обработкой надо перекодировать, либо научиться записывать данные из этой кодировки.
6. Данные из файла users(1).csv надо загрузить в таблицу USERS со столбцами user_id, email, geo.
7. Данные грязные, их надо почистить. Например, надо удалить строки с ошибками в user_id, оставить только значения вида user_N, где N - значение идентификатора.
8. Теперь надо ответить на несколько вопросов по бизнесу с помощью оконных функций:
 - а) Сумма ставок по каждому клиенту нарастающим итогом?
 - б) Каков средний выигрыш в процентах от суммы ставок по каждому клиенту?
 - в) Каков средний баланс по каждому городу?